HEdClass Report

System Overview

HEdClass is a web-based classification system designed for use by higher education institutions. It enables classification officers to manage student records, apply classification rules and generate final award outcomes. It enables admin users to manage classification officers and assign them to specific programmes.

The system automates classification calculations, supports manual overrides, and provides statistical insights for cohort analysis.

Repository & Access Details

Repository:

https://gitlab.eeecs.qub.ac.uk/40365038/hedclass

Setup Instructions:

1. Clone the repository:
   https://gitlab.eeecs.qub.ac.uk/40365038/hedclass
2. Install dependencies:
   npm install
3. Configure database connection based on XAMPP/MAMP (set to XAMPP by default)
4. Start application
   npx nodemon
5. Access in browser
   http://localhost:3000

Database Setup:

Import provided seeder file within:
/src/seeder/seed.sql

Legacy Users:

Classification Officer:

Role	Email	Password
Classification Officer (1)	officer1@hedclass.com	password123
Classification Officer (2)	officer2@hedclass.com	password123
Admin	admin@hedclass.com	password123

Version Control:

Evidence of version control can be found in gitlog.txt, which contains a summary of git commits used throughout the project, satisfying requirement D5.

Implemented Functions

Login Page

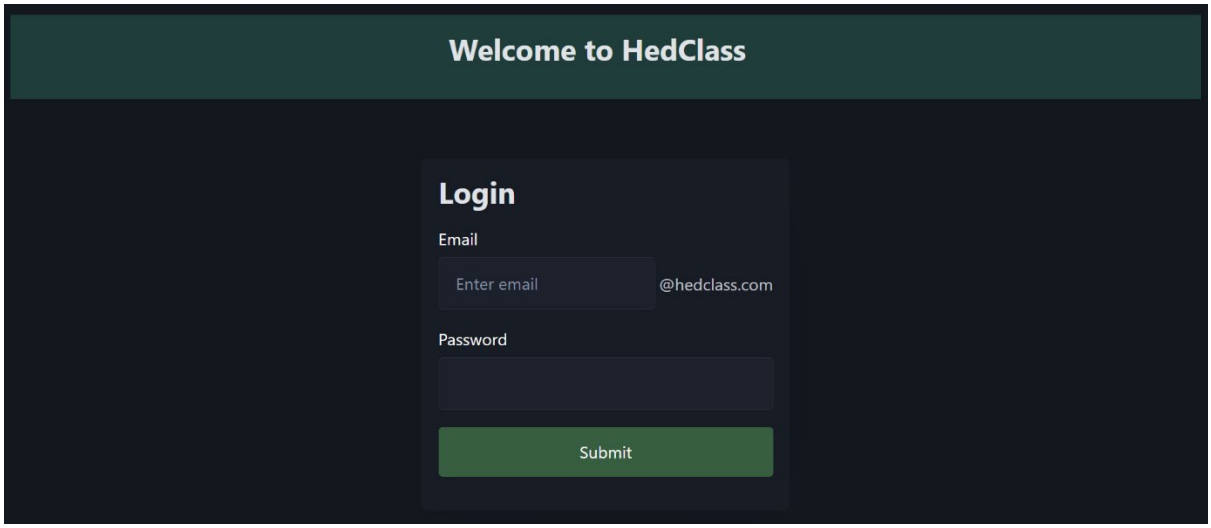

Figure 1. Login Page (*"/"*)

Login page allows user to enter their credentials and password, satisfying requirement A1.

Based on credentials entered, system will display role-specific pages by using authentication middleware, as shown in Figure 2 (satisfying requirements A3 and D3). If the user is authenticated, a session for that user will be stored to allow the system to maintain state, thus satisfying requirement A2.

src > web > middleware > JS auth.js > ...

```
export const requireAuth = (req, res, next) => {
    if (req.session.user){
        next(); //user authenticated, continue to next middleware
    } else {
        res.redirect("/"); // user not authenticated, redirect to login page
    }
}

export const requireAdmin = (req, res, next) => {
    if (!req.session?.user){
        return res.redirect("/"); //if no user session found, redirect to login page
    }

    if (req.session.user.role !== "admin"){
        return res.status(403).send("Forbidden") //if user role is not admin, send forbidden status message
    }

    next();
};

export const requireOfficer = (req, res, next) => {
    if (!req.session?.user){
        return res.redirect("/"); //if no user session found, redirect to login page
    }

    if (req.session.user.role !== "officer"){
        return res.status(403).send("Forbidden") //if user role is not officer, send forbidden status message
    }

    next();
};
```

Figure 2. Authentication middleware for role-based access

Admin Dashboard

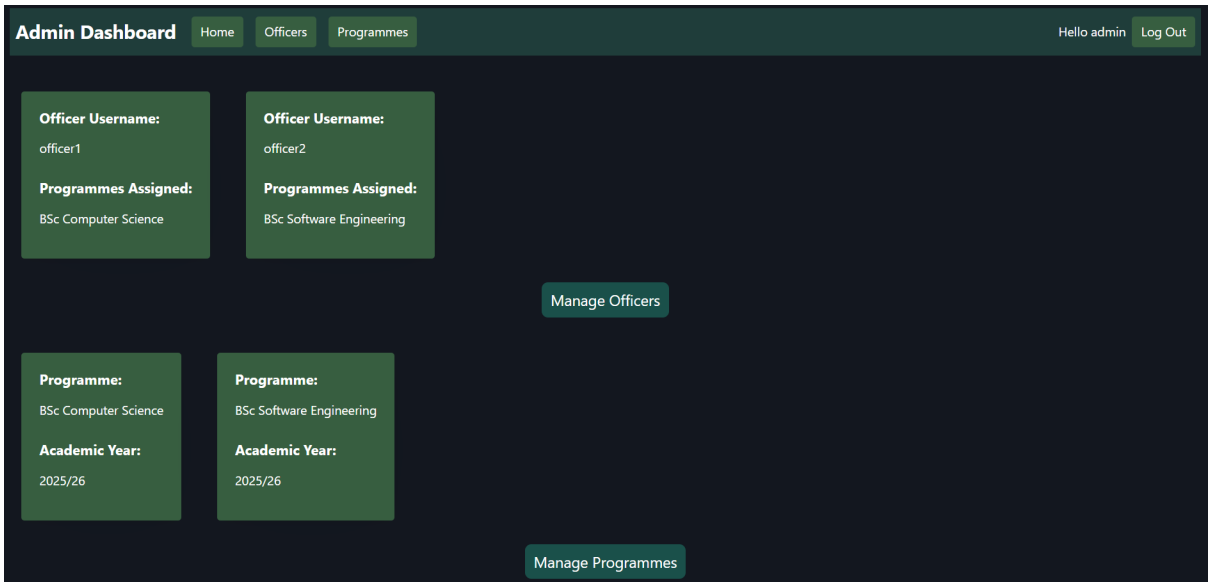

Figure 3. Admin Dashboard (*"/admin")*

Manage Officers

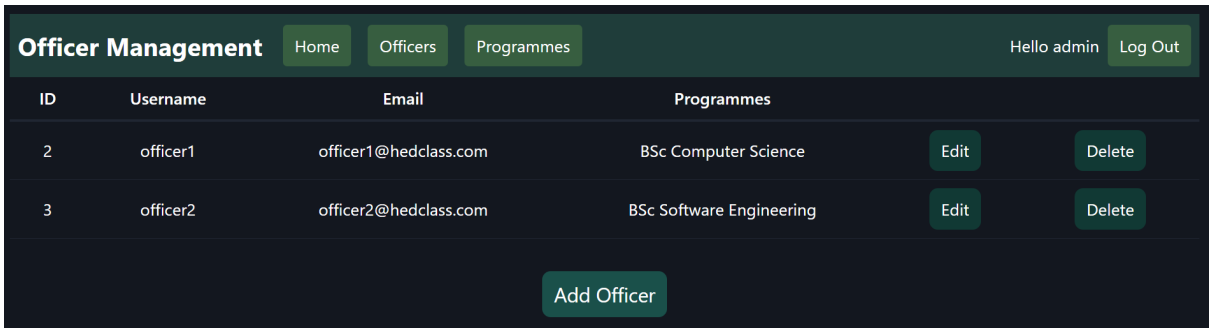

Figure 4. Officer Management page (*"/admin/officers")*

Figure 4 shows officer management page, allowing admin to edit, delete and add new officers, satisfying requirement B1.

Figure 5 shows the add officer form, showing that an officer can be assigned to multiple specific programmes, satisfying requirements B2 and B4, similar concept used within edit officer form.

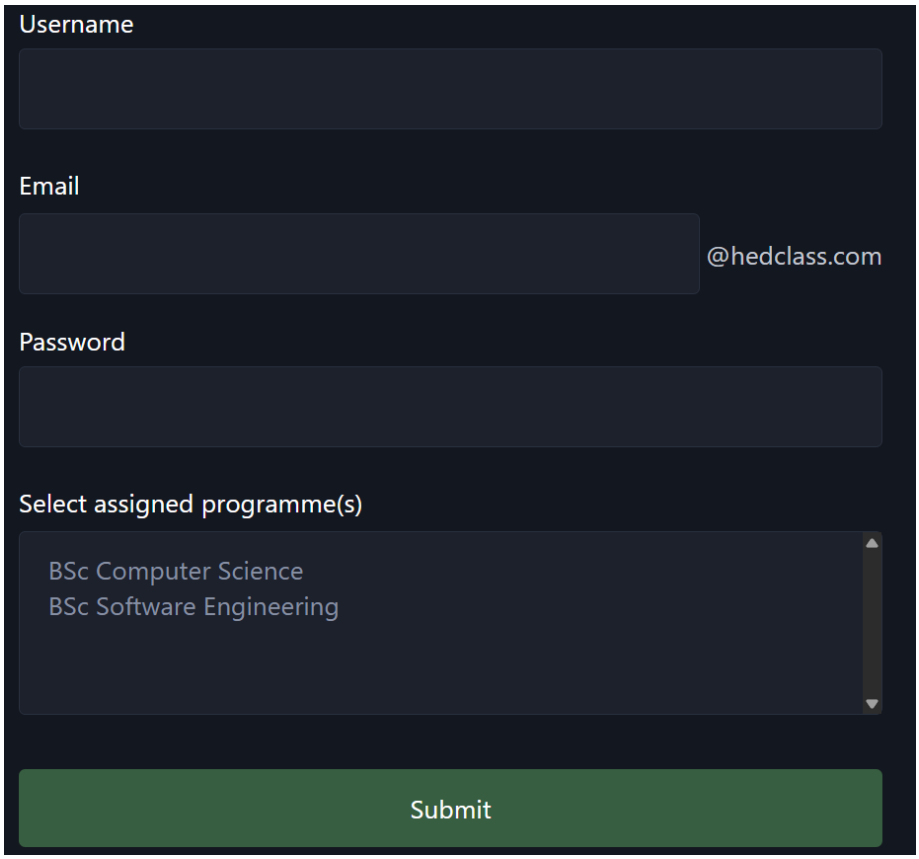

Figure 5. Add officer form (*"/admin/officer/add"*)

Manage Programmes

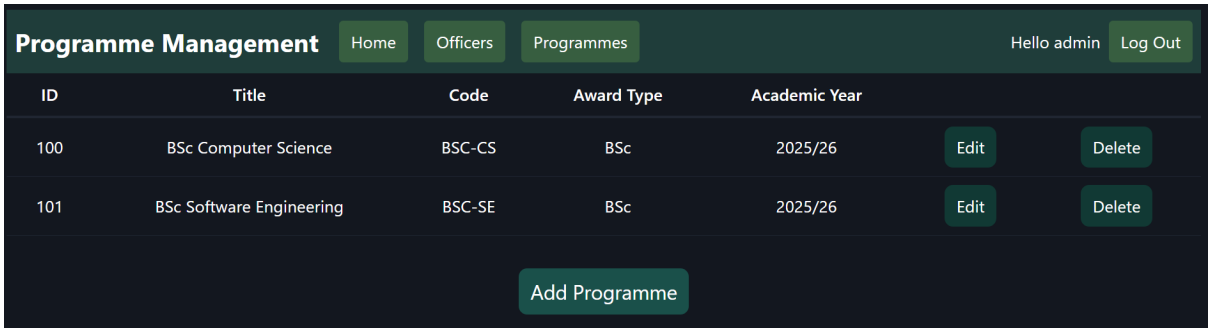

Figure 6. Programme Management page (*"/admin/programmes"*)

The programme management page allows the admin to create programme with base classification rules, satisfying requirement B3.

Officer Dashboard

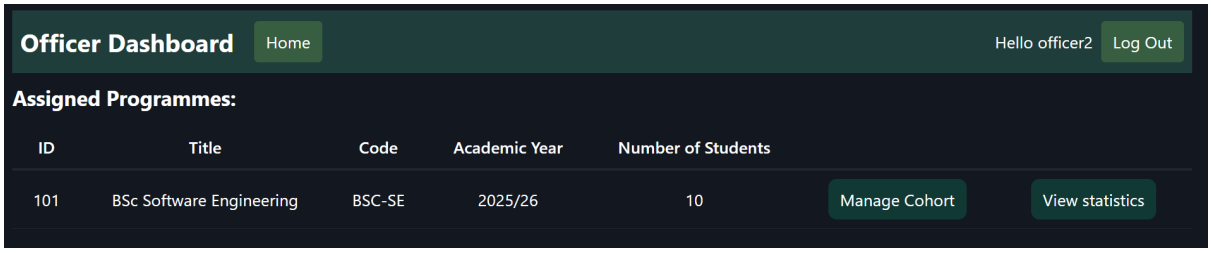

Figure 7. Officer Dashboard (*"/officer")*

The officer dashboard shows details of each programme assigned to specific officer. Selecting manage cohort will display Figure 8 page, and selecting view statistics will display Figure 10 page. This page satisfies requirement C1.

Student Management

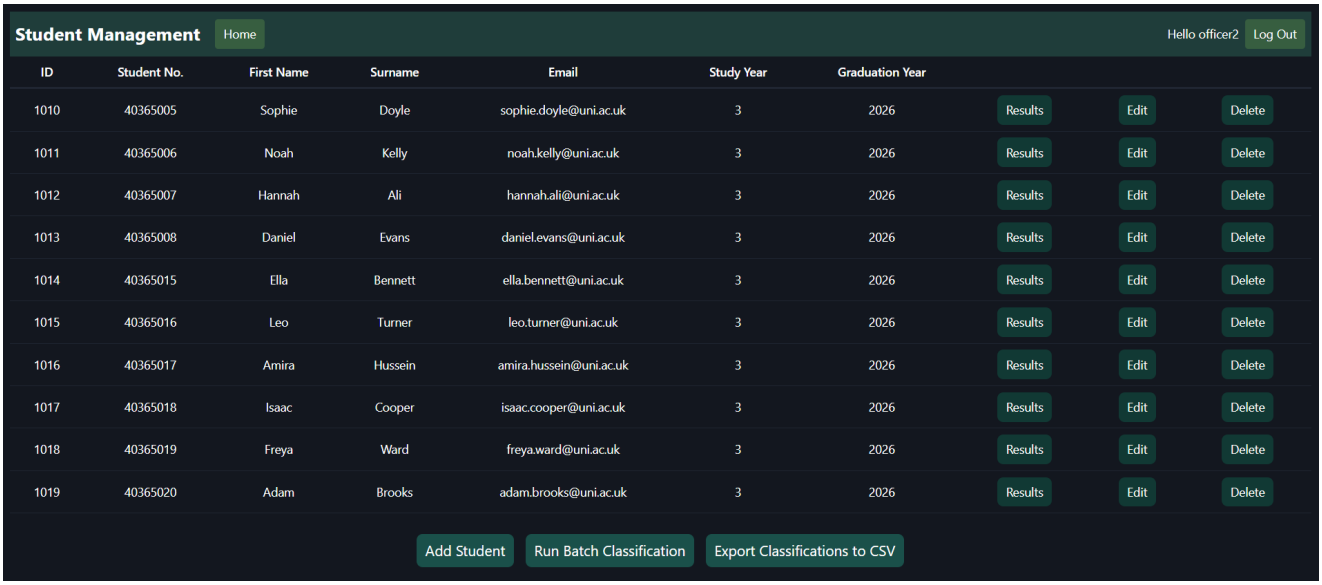

Figure 8. Student Management (*"/officer/programme/:programmeId/students")*

Figure 8 shows the cohort management page. Here, officer can view the student's results (shown in Figure 9.1), edit their details, delete their record, and add a new student, satisfying requirement C2.

The system will generate proposed classification after "Run Batch Classification" button is pressed and display a success/failure pop-up to user. The batch classification is executed via an API, which will calculate each student's final average using weighted module results. If a student has not met all classification requirements, e.g. they have an outstanding fail, the system will flag them for manual review by the officer. This satisfies requirement C3.

"Export Classifications to CSV" will download a CSV file containing each student's classification for the specific programme, satisfying requirement C7.

Student Results

Figure 9.1. Student Results (Top) (*"/officer/programme/:programmeId/student/:studentId/results"*)

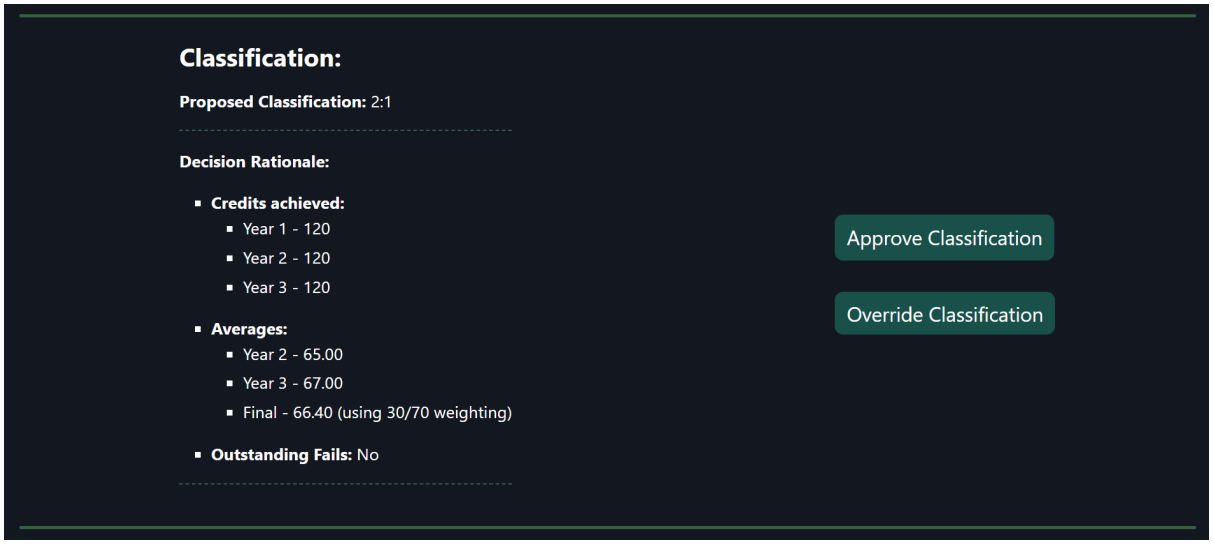

Figure 9.2. Student Results (Bottom) (*"/officer/programme/:programmeId/student/:studentId/results"*)

The student results page (Figure 9.1) will display student information and allow officer to edit this information if necessary and enable the officer to export the individual student's results to a CSV file (again satisfying requirement C7).

This page will show the student's results for each academic year they have completed, along with their total credits for that year and their average for that year. Officer may also add a result or edit/delete a particular result.
Note the use of capped mark as requirement C8, as resits will be capped at passing mark of 40.

Following batch classification, the results page will display the proposed classification generated by the system (Figure 9.2). The final average is calculated based on the year 2 and year 3 averages with a weighting of 30/70 respectively. Officer will then be able to approve classification or override the decision. If an officer chooses to override the classification they must provide an override reason, ensuring full auditability.

This section satisfies requirements C4, C5 and D6.

Programme Statistics

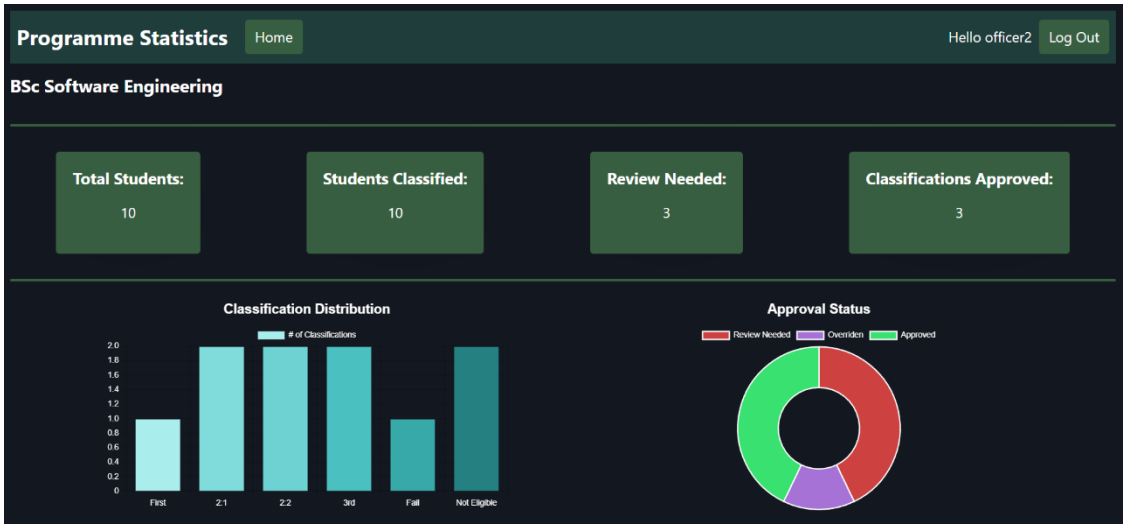

Figure 10. Programme Statistics (*"/officer/statistics/:programmeId")*

The programme statistics page shows the student count, number of students classified, how many students have been flagged for review, and how many students have been approved.

Chart.js was used to generate two statistical charts, a bar chart showing the classification distribution, and a doughnut chart showing the classification approval status.

System Architecture

For this system I have used a model-view-controller (MVC) approach, to appropriately separate concerns in my code. The models handle database queries, the controller handles the page requests, and the views handle the actual EJS rendering. This separation improves maintainability and scalability by isolating business logic from presentation logic, therefore satisfying requirement D2.

I have also made use of several JavaScript files to perform key functions, such as exporting a table to CSV format, creating my statistical charts, and storing the logic for downloading the CSV file.

Since I completed the initial web app using this structure, I did not want to refactor most of it to incorporate a separate REST API server. Instead, I chose to shift my batch classification logic into an API instead, allowing me to gain the experience working with an API, but on a more manageable level for how I approached this project.

This satisfies requirements E1 and E2.

Figure 11. VSCode folder structure

The user interface was designed using a responsive layout to ensure usability across different screen sizes, with consistent styling and spacing used to improve user experience, satisfying requirement D4.